

Manual de Referencia JPQL (Java Persistence Query Language)

1. Introducción a JPQL

JPQL (Java Persistence Query Language) es el lenguaje de consultas orientado a objetos utilizado por JPA (Java Persistence API). A diferencia del SQL tradicional, JPQL no trabaja sobre tablas y columnas, sino sobre entidades y sus atributos. Permite realizar operaciones de selección, actualización y eliminación utilizando la semántica de las clases Java persistentes.

JPQL es independiente del motor de base de datos subyacente, ya que el proveedor de JPA (como Hibernate) se encarga de traducir las consultas JPQL a SQL nativo. Esto promueve la portabilidad y la abstracción de la capa de datos en las aplicaciones empresariales.

2. Sintaxis general

Una consulta JPQL básica sigue la estructura:

SELECT <entidad_o_atributo> **FROM** <Entidad> [**AS** <alias>] [**WHERE** <condición>]

Ejemplo:

```
SELECT c FROM Cliente c WHERE c.edad > 30
```

3. Consultas de selección (SELECT)

Permiten recuperar entidades o valores específicos.

- SELECT e FROM Entidad e → devuelve todas las instancias.
- SELECT e.nombre FROM Entidad e → devuelve solo el atributo nombre.
- SELECT DISTINCT e.ciudad FROM Entidad e → evita duplicados.
- SELECT NEW com.ge.dto.ClienteDTO(e.nombre, e.edad) FROM Cliente e → crea instancias DTO.

4. Filtros y condiciones (WHERE)

La cláusula WHERE restringe los resultados mediante expresiones lógicas.

Operadores soportados: =, <>, <, >, <=, >=, BETWEEN, LIKE, IN, IS NULL, AND, OR, NOT.

Ejemplo:

```
SELECT p FROM Persona p WHERE p.nombre LIKE 'J%' AND p.edad BETWEEN 20 AND 40
```

5. Parámetros en consultas

Se pueden usar parámetros posicionales (?1, ?2) o con nombre (:nombre).

Ejemplo con parámetro con nombre:

```
SELECT c FROM Cliente c WHERE c.nombre = :nombre
```

```
em.createQuery(...).setParameter("nombre", "Jorge").getResultList();
```

6. Joins en JPQL

Permiten combinar entidades relacionadas según sus asociaciones.

- INNER JOIN: devuelve solo coincidencias.
- LEFT JOIN: devuelve todas las entidades de la izquierda aunque no haya relación.
- JOIN FETCH: se usa para cargar relaciones LAZY de forma anticipada.

Ejemplo:

```
SELECT c FROM Cliente c JOIN FETCH c.direccion d WHERE d.provincia = 'Mendoza'
```

7. Funciones y operadores

Funciones de agregación: COUNT, SUM, AVG, MIN, MAX.

Funciones de texto: CONCAT, SUBSTRING, LOWER, UPPER, LENGTH.

Funciones matemáticas: ABS, SQRT, MOD.

Ejemplo:

```
SELECT AVG(p.edad) FROM Persona p WHERE p.ciudad = 'Mendoza'
```

8. Agrupación y ordenación

Se usan para resumir datos y ordenar resultados.

- GROUP BY agrupa registros.
- HAVING filtra resultados agrupados.
- ORDER BY ordena resultados.

Ejemplo:

```
SELECT d.provincia, COUNT(d) FROM Direccion d GROUP BY d.provincia HAVING  
COUNT(d) > 10
```

9. Subconsultas

Permiten incluir consultas dentro de otras.

Ejemplo:

```
SELECT c FROM Cliente c WHERE c.id IN (SELECT p.cliente.id FROM Pedido p WHERE  
p.monto > 1000)
```

10. Consultas de actualización y borrado

JPQL permite actualizar o eliminar datos mediante UPDATE y DELETE.

Ejemplo UPDATE:

```
UPDATE Cliente c SET c.estado = 'INACTIVO' WHERE c.saldo > 5000
```

Ejemplo DELETE:

```
DELETE FROM Pedido p WHERE p.fecha < '2023-01-01'
```

11. Named Queries

Las Named Queries se definen en las entidades usando anotaciones y se reutilizan desde el código.

Ejemplo:

```
@NamedQuery(name="Cliente.findAll", query="SELECT c FROM Cliente c")  
@NamedQuery(name="Cliente.findByName", query="SELECT c FROM Cliente c WHERE  
c.nombre = :nombre")
```

Uso:

```
em.createNamedQuery("Cliente.findAll", Cliente.class).getResultList();
```

12. Relación con EntityManager

El EntityManager ejecuta las consultas JPQL a través de sus métodos principales:

- createQuery(String qlString, Class<T> resultClass)

- setParameter(String name, Object value)
- getResultList()
- getSingleResult()
- getUniqueResult()

Ejemplo:

```
List<Cliente> clientes = em.createQuery("SELECT c FROM Cliente c",  
Cliente.class).getResultList();
```

13. Buenas prácticas y errores comunes

- Usar siempre alias en las consultas.
- Preferir Named Queries para evitar errores de sintaxis en tiempo de ejecución.
- Utilizar JOIN FETCH con precaución para evitar sobrecarga de datos.
- Manejar correctamente las excepciones NoResultException y NonUniqueResultException.
- Recordar que las consultas JPQL se ejecutan sobre entidades, no sobre tablas.